

Thunk

A Peer-to-Peer Evaluator for Pure Functional Computations Final Report for the Distributed Systems Course, academic year 2025/2026

Yosberto Baro Carbonell
`yosberto.baro@studio.unibo.it`

September 28, 2026

Thunk is a distributed runtime that evaluates pure functional computations on a cluster of peer nodes, without a central coordinator. Computations are expressed in a small functional language whose only parallel construct is a divide-and-conquer primitive, map and reduce are then built on it. Programs travel between nodes as data, so a node needs no prior knowledge of the programs it helps to run. Work is distributed by randomized work stealing and, since computations are pure, a piece lost with a failed node is simply recomputed by the node waiting for it: crashes are tolerated without replication. It was validated with automated tests on real additional nodes and with experiments on one machine and on two machines connected over the Internet.

1 Concept

1.1 Product Description

This project implements a *distributed evaluator for pure functional computations* in the form of a library and a set of cooperating nodes. The system is used programmatically, by submitting a program and an input value on any node of the cluster, and it is observed through its command-line output and logs.

The primary use case is the execution of computations that are naturally recursive and data-parallel, such as sorting, counting or rendering, on a small group of machines where:

- no machine should play a special role;
- machines may fail while a computation is running;
- the computation is not known in advance by the machines that will help run it.

Users write a program in the Thunk language, start a job from a shell on one of the nodes, and wait for its result. Interaction is occasional: a job runs for seconds or minutes and returns a single value. There is no hierarchy, every node is equally able to submit

jobs, compute pieces of them, and help other nodes. No persistent data is stored: a program, its input and all intermediate results exist in memory only for the duration of the job.

The system follows a *fork-join* model with *randomized work stealing*: a node that splits a problem keeps one half and exposes the other, and idle nodes take exposed work from randomly chosen peers. Since the language is pure, the result never depends on where or when each piece was computed, and a piece lost with a failed node can simply be computed again.

1.2 Why Distribution Is Needed

- **Computation speed-up:** a divide-and-conquer computation can be split into many independent pieces, and the processors of several machines can work on them at the same time.
- **Resource sharing:** idle machines contribute their processing time to jobs submitted elsewhere, without having to know the program in advance.
- **Fault tolerance:** a centralized scheduler would be a single point of failure; without one, the loss of a machine only loses the work it was doing.
- **Educational value:** a practical study of scheduling without a coordinator, of moving computations as data between machines, of failure detection between remote processes, and of the relation between purity and fault tolerance.

2 Requirements Elicitation and Analysis

2.1 Glossary

- **Program:** a sequence of top-level definitions written in the Thunk language. Running a program means applying its definition called `main` to an input value.
- **Prelude:** the standard library of the language, itself written in the language and available on every node.
- **Job:** one execution of a program on an input, submitted on a node called its *origin*.
- **Divide and conquer (dc):** the only parallel construct of the language (section 3.3).
- **Piece:** a part of a job waiting to be computed: a value together with the functions that say how to split it, solve it and combine the results.
- **Owner:** the process that split a piece off and is waiting for its result.
- **Thief:** a node that takes a piece from another node.
- **Threshold:** the criterion that decides when a value is small enough to be solved directly. In Thunk it is a predicate written by the user.

2.2 Functional Requirements

Each requirement is followed by its acceptance criterion.

- **FR1, Functional Language:** users must be able to express computations as programs in a pure functional language offering map, reduce and a divide-and-conquer primitive. *Acceptance:* the library functions and the demonstration programs return the same results as equivalent native implementations.
- **FR2, Recursive Decomposition:** a divide-and-conquer computation must be split recursively until the user's threshold states that a part is small enough; the parts must then be solved and their results combined in order. *Acceptance:* splitting stops exactly where the threshold says, results are combined in the order of the split, and misuse (a threshold that does not return a boolean, a split that does not produce two parts) is reported as an error.
- **FR3, Decentralized Work Stealing:** any node must be able to submit a job; pieces of the job must be taken and computed by other nodes, and their results combined into the result of the job. *Acceptance:* a job submitted on a node that is not allowed to compute pieces itself returns the correct result, and the other nodes report stolen pieces.
- **FR4, Program Distribution:** a program must only have to be known by the node that submits it; the other nodes must obtain it when they first need it. *Acceptance:* a job whose program uses definitions that exist only on the submitting node completes correctly on the cluster.
- **FR5, Error Reporting:** an error in the program, wherever it occurs, must be reported to the submitter as the same error a sequential execution would have produced. *Acceptance:* a program that fails in a piece computed by a remote node raises the same error at the submitter.
- **FR6, Recovery from Node Failures:** a job must complete with the correct result if a node that holds pieces of it crashes or disconnects while it runs. *Acceptance:* after intentionally stopping a node during a job, the job returns the same result it returns without the failure.
- **FR7, Demonstration Workloads:** word count and mergesort must be runnable from the command line on one node or on the cluster, with a check of the result. *Acceptance:* every run of the demonstrations reports a correct result under every scheduling mode.
- **FR8, Dynamic Membership (extension):** a node must be able to join a running cluster knowing a single existing node, and nodes that leave must be forgotten, without any central register. *Acceptance:* a node started with one seed ends up connected to every node of the cluster, and a node that joins while a job runs takes part in that job.

2.3 Non-Functional Requirements

- **Determinism:** the result of a job must not depend on which node computed which piece, on the number of nodes, or on the order in which pieces finish.
- **Decentralization:** no node may have a special role in scheduling.
- **Liveness under failure:** a job must never wait forever because a remote partic-

ipant failed.

- **Low distribution overhead:** on the same hardware, running a job across nodes must cost little compared with running it on local processes only.
- **Configurability:** the peers of a node and the amount of work it performs locally must be configurable through environment variables.
- **Observability:** each node must expose counters of stolen, given, evaluated and recovered pieces, and the state of the whole cluster must be observable live while a job runs.

2.4 Implementation Requirements

The system is implemented in Elixir on the Erlang virtual machine (BEAM), as agreed in the project proposal: the platform provides lightweight processes, message passing between machines and failure notification between remote processes. Nodes communicate through the native distribution of the Erlang runtime, without external middleware, and Docker Compose runs a multi-node cluster reproducibly. No third-party libraries are used, so that every distributed mechanism is implemented and studied directly.

2.5 Relevant Distributed System Features

- **Transparency:** location transparency is central. A program cannot observe where its pieces run, and the same program runs unchanged on one process, on several local processes or on several nodes. Failure transparency is provided for node crashes, which are hidden by recomputation, but intentionally not for errors in the program, which are reported.
- **Fault Tolerance and Dependability:** the system tolerates crash failures of the nodes that help compute a job, not of the node that submitted it. It holds no data that could be lost, only computation, which purity makes repeatable (section 3.7).
- **Scalability:** the system targets a handful of machines on a local network. Work stealing only requires each node to know its peers, and the amount of coordination grows with the number of stolen pieces, not with the size of the input.
- **Consistency:** there is no replicated state; the only shared data, the definitions of a job, are immutable, and consistency of the results follows from purity (section 3.6).
- **Resource Sharing:** the shared resource is processor time. Idle nodes pull work from busy ones.
- **Maintainability and Evolvability:** the language has only thirteen primitive operations; everything else, including map, reduce and sorting, is library code written in the language itself, so the library can grow without changing the runtime.
- **Performance and Efficiency:** the relevant metrics are the time to complete a job, the overhead of distribution compared with local execution, and the amount of data carried by each piece of work, since pieces may cross the network.
- **Not goals:** security, since the system assumes a trusted private network (section 3.9); openness, since all nodes run the same software and speak the native

protocol of the runtime; and cost, since the system runs on machines the user already has and uses no paid service.

3 Design

3.1 Architecture

The system adopts a fully decentralized *peer-to-peer* architecture. Every node is architecturally identical: it can accept a job, split work, compute pieces and give pieces to others. There is no master, no central queue and no shared database.

Within a job, computation follows the *fork-join* model with *randomized work stealing* introduced by the Cilk runtime [5] and analysed by Blumofe and Leiserson [1]. A process that splits a problem keeps one half to work on and exposes the other in a deque; nodes with nothing to do steal from the deques of other nodes, choosing the victim at random. This choice is motivated by:

- **No single point of failure:** a master that assigns tasks, as in MapReduce-like systems [2], would be a component every job depends on.
- **No global knowledge:** random stealing only requires each node to know its peers, and it is known to balance load efficiently with few steal attempts [1].
- **Local bookkeeping:** whoever splits a piece waits for its result. No component keeps a table of the pending tasks of a job, and the end of a job is simply the return of its first computation (section 3.5).

Two alternatives were discarded. An *explicit task tree*, in which the submitting node records every piece and receives completion notices, gives cleaner accounting but turns that node into a bookkeeper for the whole job, close to a coordinator. A *shared store*, such as a message queue holding pieces and results, would route all communication through a central component, give every piece an identity and every result an address, and make serialization a concern of the application.

3.2 Infrastructure

The system is composed exclusively of *homogeneous nodes*, each an instance of the Erlang virtual machine running the Thunk application (fig. 1). Each node includes four logical components:

1. **Worker:** a long-lived process that holds the node's deque of stealable pieces and a cache of the programs of the jobs it has met, and decides when to steal.
2. **Evaluators:** short-lived processes, each computing one piece by interpreting the program.
3. **Evaluator supervisor:** starts evaluators and isolates their failures, so that a failing program cannot bring the worker down.
4. **Membership process:** keeps the list of the other nodes and connects to the ones it learns about (section 3.5).

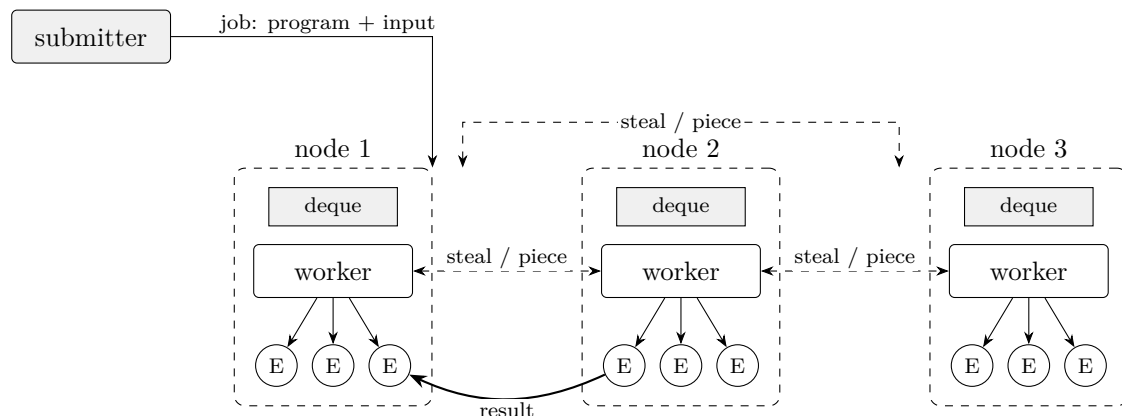


Figure 1: Decentralized architecture of a Thunk cluster. Every node has the same structure: a worker holding a deque of pieces, and evaluator processes (E) that compute pieces. Workers exchange steal requests and pieces; the result of a stolen piece is sent directly to the process waiting for it.

A client is not a separate component: submitting a job means calling the library on any node, which then acts as the origin of the job. In the demonstrations, the submitting node is an additional short-lived node started next to the others.

Nodes are meant to run on different machines of a local network; in the Docker Compose deployment they run as containers on a shared virtual network. Each node is identified by a name of the form `name@host`. A node is started with a list of *seeds*, names of existing nodes passed through an environment variable, and needs only one of them to be reachable; it then discovers the rest of the cluster by gossip. The Erlang name service on each host maps node names to ports, and in Docker Compose host names are resolved by the container DNS.

3.3 Modelling

The language. The language is pure, strict, lexically scoped and dynamically typed. Its values are integers of arbitrary size, booleans, strings, lists and functions. A program is a sequence of definitions, and everything in it is an expression: `fn` builds a function, `let x = e in body` binds one name, `if c then a else b` chooses a branch, and `f(x, y)` is a call. There are thirteen primitive operations: arithmetic and comparison on integers, construction and inspection of lists, and conversion between strings and lists of character codes; the infix operators `+` `-` `*` `/` `%` `<` `==` `::` are only a notation for them. Everything else, from list functions to sorting and the parallel operations, is written in the language itself, in the prelude.

```
def isqrt_iter(n, x) =
  let y = (x + n / x) / 2 in
  if y < x then isqrt_iter(n, y) else x
```

The parser is generated from a grammar, in the tradition of Yacc [6], and turns a

program into nested lists, the representation of programs in Lisp [7]: the definition above becomes a list headed by **def**, its body a list headed by **let**, and each operator a call of its primitive. The interpreter and the runtime only ever see these lists, and so do the pieces of work that travel between nodes.

The only parallel construct is

$$dc(value, small?, split, base, merge)$$

whose meaning is

$$solve(v) = \begin{cases} base(v) & \text{if } small?(v) \\ merge(solve(l), solve(r)) & \text{otherwise, where } [l, r] = split(v) \end{cases}$$

The threshold is part of the user's predicate, so the runtime needs no notion of size. The merge always receives the left result first, so the result does not depend on where each half was computed or which one finished first. Parallel map and reduce are ordinary definitions:

```
def pmap(f, xs, small?) = dc(xs, small?, halves, pmap_base(f), append)

def reduce(op, xs, small?) = dc(xs, small?, halves, reduce_base(op), op)

def mergesort(xs, small?) = dc(xs, small?, halves, insertion_sort, merge_sorted)
```

Parallelism is therefore never inferred by the runtime: choosing **reduce** instead of the sequential **fold** is how a user states that an operation is associative and may be computed in parallel.

Domain entities. A *program* is a table of named definitions, obtained by parsing the source and evaluating each definition; a *job* is identified by a unique reference and by its origin node, which keeps the program. A *work* is a value together with the four functions of the **dc** that produced it; functions are closures, that is, data: the parameter names, the body and the captured variables. A *piece* is a work together with a unique reference, the address of the process waiting for its result, and the identity of its job. All entities are held in memory by the nodes that use them (fig. 2). A piece lives in exactly one place at a time: in the deque of the node that created it, in the hands of the evaluator computing it, or nowhere once solved.

Domain events. Job Submitted; Piece Pushed (a right half becomes available); Piece Stolen, or Taken Back by its owner; Piece Claimed (a thief started computing it); Result Returned; Piece Lost (its evaluator or node failed); Program Fetched (a node obtains the definitions of a job).

Messages. Table 1 lists the messages exchanged between processes. Apart from fetching a program, which is a request followed by a reply, all messages are asynchronous.

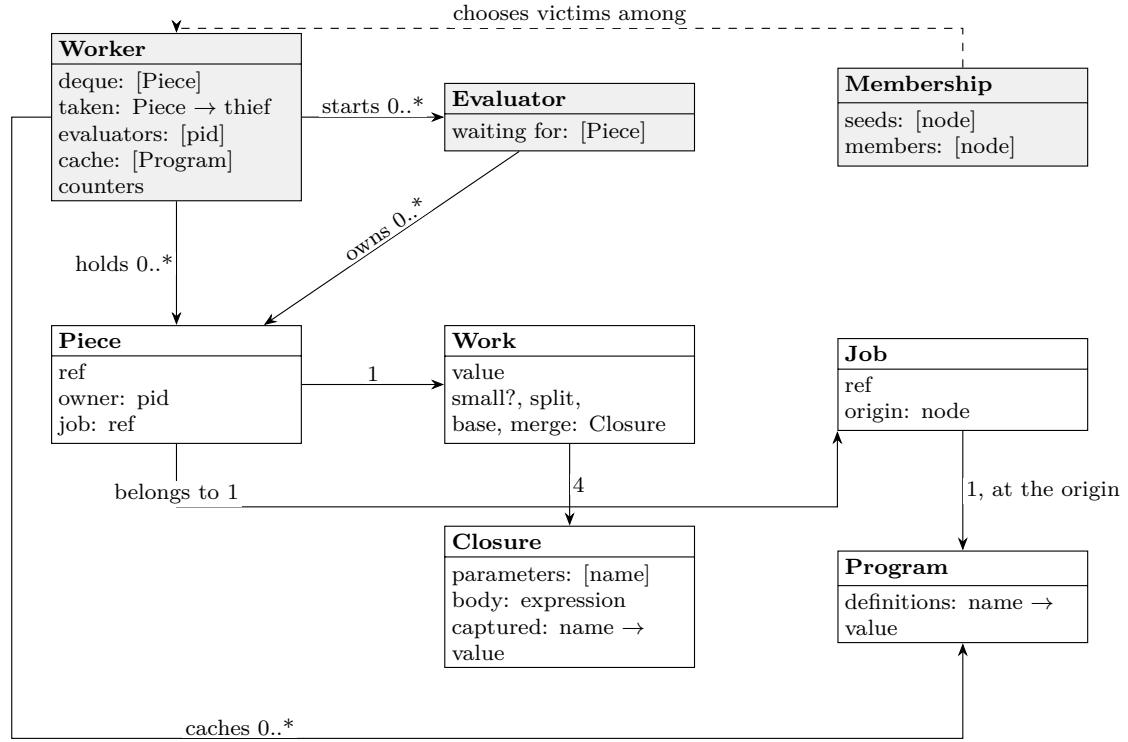


Figure 2: Domain entities and the components that hold them. Grey boxes are processes, white boxes are immutable data. A piece points to its work and to the evaluator waiting for it; the four functions of a work are closures, so a piece carries whatever they captured.

Message	From	To	Meaning
push	evaluator	local worker	a right half is available
take back	evaluator	local worker	the owner wants its piece back; the reply is the piece, or the worker that took it
steal	worker	remote worker	request for the oldest piece
piece / none	worker	thief	reply to a steal request
claimed	thief worker	owner	the piece is being computed by a given evaluator
result	evaluator	owner	the value of a stolen piece
failed	thief worker	owner	the evaluator of a piece died, with the reason
get job	evaluator	origin worker	request for the program of a job
pull	membership	random peer	request for the peer's list of members
push	membership	asking node	the list of members, to connect to the unknown ones

Table 1: Messages exchanged between Think processes.

State. The state of a node is the state of its worker (fig. 2); the state of a job in progress is spread over the evaluators computing it, each remembering the pieces it has split off and is waiting for. There is no global state.

3.4 Interaction

Nodes interact through the steal protocol shown in fig. 4:

1. An evaluator that splits a value pushes the right half to its node's worker and continues with the left half.
2. A worker with spare capacity and an empty deque sends a steal request to a peer chosen at random.
3. The victim answers with its oldest piece, or with *none*.
4. The thief's worker starts an evaluator for the piece and tells the owner which process is computing it.
5. When the owner has finished its own half, it asks its worker for the right half back. If nobody took it, the owner computes it itself; otherwise it waits for the result.
6. The thief's evaluator sends the result directly to the owner, which combines it with its own.

The victim gives away its *oldest* piece, while a node's own evaluators take the *newest*. In a divide-and-conquer tree the oldest piece is the largest: one network round trip transfers the most work, and the thief can split it further and become self-sufficient. The newest piece is the smallest, and its data is already local. Figure 3 shows why on a list of sixteen elements.

From a communication perspective, a stolen piece costs a request, a reply, a claim and a result: four messages, independently of the size of the job. The amount of data they carry, however, depends on what the piece contains, which is discussed in section 3.6.

3.5 Behaviour

Evaluators. An evaluator is stateless apart from the pieces it is waiting for. When the interpreter meets a `dc`, the evaluator follows the logic of Listing 1. The same logic runs in the evaluator of a stolen piece, which is how a thief splits a large piece further and exposes its own halves to other nodes. If an error interrupts the computation, each level first withdraws its own piece from the deque, so that a failed job leaves no work behind.

Termination. A job ends when the evaluation of `main` returns. Since every piece has an owner that waits for it before returning, the return of the first computation implies that every piece has been accounted for. No termination detection protocol is therefore needed: the Dijkstra–Scholten algorithm [3], listed among the optional extensions of the proposal, becomes unnecessary in this model. The distinction between “result available”, “no local work” and “global termination” collapses: the first is what the submitter observes, the second is a local condition used by workers to decide when to steal, and the third never has to be detected.

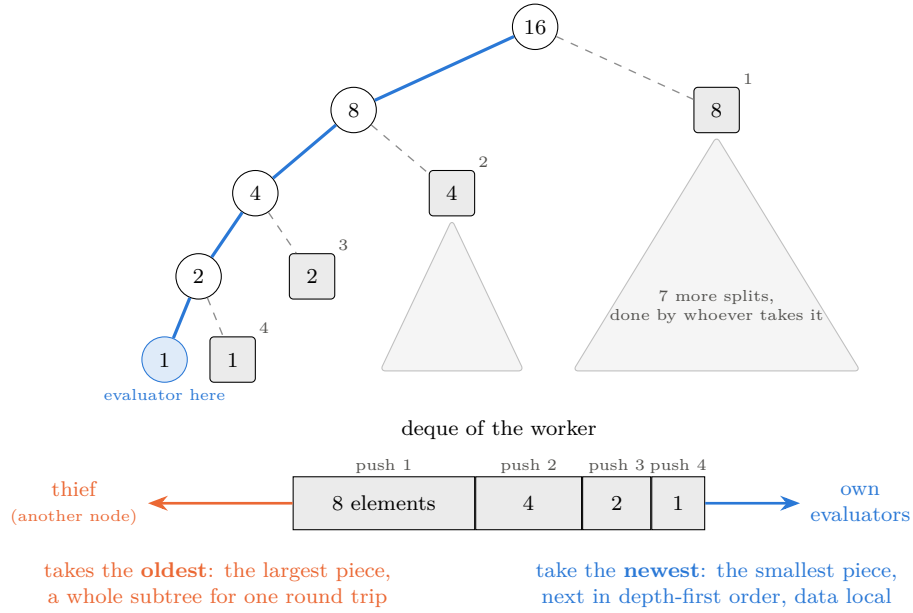


Figure 3: The two ends of the deque. Numbers are sizes in elements. An evaluator descends along the left half at every level and pushes the right half, so the deque holds pieces of decreasing size in push order. A thief takes the oldest piece, a whole subtree that it splits further on its own node; the node’s own evaluators take the newest, which is the next leaf in depth-first order.

```

solve(work):
  if small?(work.value): return base(work.value)
  [left, right] = split(work.value)
  push the right half as a new piece
  l = solve(left)
  case take_back(right piece):
    still in the deque -> r = solve(right)
    stolen by a thief -> wait for the result:
      result v -> r = v
      error of program -> raise it again
      evaluator/node lost -> r = solve(right)
  return merge(l, r)

```

Listing 1: Simplified logic of an evaluator for a divide-and-conquer step.

Workers. The worker is the only stateful component, and it changes state only in response to messages, one at a time (fig. 5).

- It starts evaluators for its own newest pieces while it has fewer running than a limit, by default the number of processor cores. Pieces beyond the limit stay in the deque, visible to thieves.
- It steals only when its deque is empty and it has spare capacity, with one request in flight at a time.

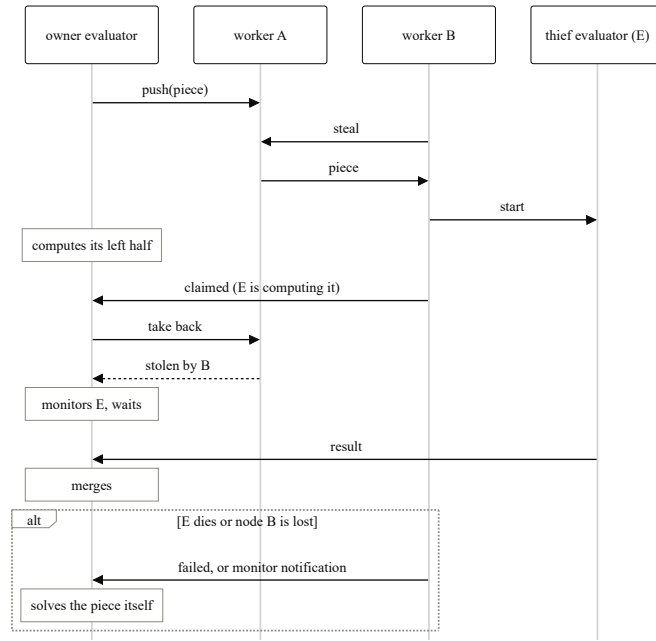


Figure 4: Sequence diagram of a stolen piece. If the owner asks for its piece back before anyone steals it, it simply computes it. In the failure case, an error of the program is reported instead of recomputing, since it would occur again.

- After an unsuccessful request it waits before asking again, doubling the wait from 10 ms up to 100 ms with random jitter, and resetting it as soon as it obtains work.
- A stolen piece always gets an evaluator, even beyond the limit, since the node asked for it.

The limit is essential to the design: a node that started an evaluator for every piece as soon as it was pushed would never leave anything in its deque, and no work could ever be stolen.

Membership. The membership process of a node tries its seeds, once a second, until it knows at least one member. From then on it spreads the member list by gossip: when it connects to a node, and every two or three seconds with random jitter, it asks a random connected node for its list and connects to the nodes it did not know. This is an anti-entropy exchange: every node eventually knows every other, without a register and without relying on the automatic full mesh of the Erlang runtime, which the two-machine deployment disables. A node that disappears is dropped from the list; a restarted node joins again through its seeds. Since thieves choose victims among the connected nodes, a node that joins while a job runs starts stealing from it at once.

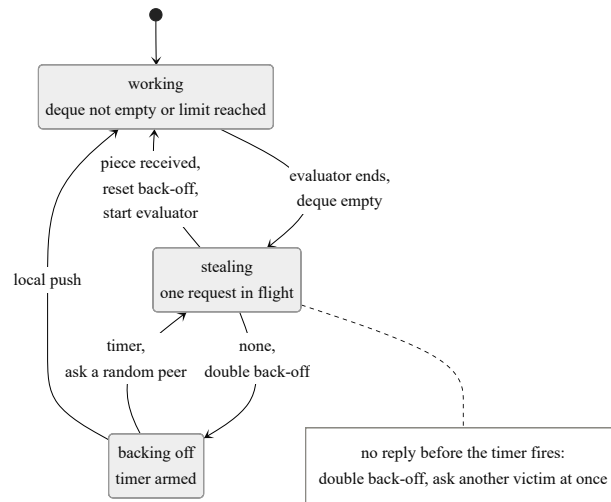


Figure 5: Finite state machine of the worker’s steal loop. In every state the worker also answers its own evaluators, steal requests from other nodes and requests for programs; a steal request that never gets an answer, because the victim failed, is treated as *none* when the timer fires.

3.6 Data and Consistency Issues

The system keeps no persistent data (section 1). The only data shared between nodes is the program of a job. It is immutable once the job is submitted, and each node caches it the first time it needs it, by asking the origin. Every copy is therefore identical by construction, and no consistency protocol is needed.

Pieces are never shared: at any time a piece is in its creator’s deque, held by exactly one evaluator, or finished. All transitions happen inside the worker, which handles one message at a time, so a piece cannot be taken back by its owner and given to a thief at the same time.

Consistency of the *results* comes from purity rather than from coordination. Whichever node computes a piece, and however many times, it produces the same value, and the order of combination is fixed by the split. The result of a job is therefore always equal to its sequential evaluation.

Since functions travel as closures, a piece carries every variable its functions captured: a function created where the input of the job is visible carries the whole input with every piece, even if it never uses it (section 4).

3.7 Fault-Tolerance

Fault tolerance is achieved through recomputation rather than replication.

- **Unit of recovery:** the piece. The owner of a stolen piece still holds its data, and purity makes computing it again safe.

- **Failure detection:** process monitors, which the Erlang runtime delivers across machines. A monitor reports the death of a remote process, with its reason, or the loss of the connection to its node, detected by the runtime's distribution layer; no application-level heartbeat or timeout is used.
- **Reporting the cause:** the thief's worker, which monitors its evaluators anyway, sends the owner the reason why an evaluator died. This is needed because owners usually look at a stolen piece only after finishing their own half, when a monitor on an already dead process can only report that it no longer exists.
- **Program errors are not retried:** if a piece failed because of an error in the program, the owner raises the same error, since a deterministic program would fail again.
- **Crashes are retried once, locally:** for any other cause, a crashed process or a lost node, the owner computes the piece in its own process. A second failure then propagates normally, so recovery cannot loop.
- **Isolation:** evaluators are not linked to their worker, so a failing program never brings a node down.

The failure of the submitting node, and Byzantine failures, are outside the scope of the project.

3.8 Availability

Each node caches the program of a job the first time it meets a piece of that job, so the origin is asked at most once per node and per job. Work stealing is the load balancing mechanism: busy nodes are never interrupted, idle nodes pull work from randomly chosen peers. In the presence of a **network partition**:

- workers only choose victims among the nodes they can reach, so each side continues with the work it can see;
- pieces held by thieves on the other side are treated by their owners as lost, and recomputed;
- the side that holds the submitting node completes the job, possibly more slowly, and results computed on the other side are discarded.

In terms of the CAP theorem, a job gives up speed rather than correctness: it never returns a partial or wrong result, and it keeps making progress as long as the submitting node is alive.

3.9 Security

Security is intentionally outside the scope of this project, and the system assumes a trusted private network: the only authentication is the shared secret (*cookie*) required by the Erlang runtime to connect two nodes, there is no authorization and there are no roles, and traffic is not encrypted.

Pieces of work are data interpreted by a language that cannot perform any action other than computing a value, so a piece cannot by itself execute arbitrary code on a

thief. The Erlang distribution layer, however, grants a connected node full control over the others, so the cookie must be treated as a password and the cluster must not be exposed to untrusted networks.

4 Implementation

- **Network Protocol:** all communication uses the distribution protocol of the Erlang runtime over TCP [4]. It provides reliable, ordered delivery between each pair of processes, and failure notifications between processes on different machines, which is what the steal protocol and the recovery mechanism rely on.
- **Data Representation:** messages are Erlang terms, encoded by the runtime in its external term format. Parsed programs are nested lists of symbols, and closures are tuples of parameter names, body and captured variables. Since no compiled code ever crosses a node boundary, the project needs no serialization code.
- **Persistence and authorization:** none; all state is in memory, and the only authentication is the runtime cookie (section 3.9).

Mapping of the design.

- **Worker:** an OTP `GenServer` registered under a fixed name, so that the worker of any node is reachable by its name and the name of the node. All exchanges between workers are asynchronous: a worker never waits for another node, and is therefore always free to answer its own evaluators. The only synchronous remote request, fetching a program, is made by evaluators.
- **Evaluators:** processes started under an OTP `Task.Supervisor`.
- **Membership:** a separate `GenServer` that exchanges member lists with plain messages; connection attempts run in separate processes, since connecting to an unreachable node can take seconds.
- **Dashboard:** a collector that asks every node for its counters twice a second, with a short timeout so that a lost node cannot block it, and a small HTTP server written directly on TCP sockets, which serves a page and the collected data as JSON. A node that stops answering is kept on the page and marked as down, so failures stay visible.
- **Schedulers:** the interpreter delegates each `dc` to a scheduler. Three schedulers share the same definition of `dc`: a *sequential* one, used as the reference for correctness; a *local* one, which runs halves on separate processes of the same node; and the *distributed* one described in section 3.
- **Interpreter:** a tree-walking interpreter. Calls in tail position are executed without growing the stack, so loops written as recursive functions run in constant memory.

Size of pieces. Since closures carry the variables they captured (section 3.6), the functions that the library passes to `dc` are created in definitions where only what they need is visible. The difference was measured on a parallel map over 100 000 integers: when

Parameter	Description	Default
node name	unique name of the node (<code>--sname</code> or <code>--name</code>)	required
cookie	shared secret of the cluster (<code>--cookie</code>)	required
THUNK_PEERS	comma-separated seeds: existing nodes to join through; one reachable seed is enough	none
THUNK_LIMIT	evaluators the node runs from its own deque	processor cores

Table 2: Configuration of a node.

the base case of `pmap` was created inside its body, where the input list was visible, every piece carried the whole input, about 500 kB; created in a separate definition it carries 341 bytes, and the job on one node became a third faster. Functions written by the user inside `main` still capture its argument, which is the input of the job, so programs should define the functions they pass to parallel operations at top level (section 7).

4.1 Technological Details

The system runs on Elixir 1.20 and Erlang/OTP 29. Besides the OTP behaviours and the Erlang distribution described above, it uses the OTP `peer` module to start real additional nodes during automated tests, ExUnit as testing framework, and Docker Compose for the containerized deployment. Nodes are configured on the command line and through environment variables (table 2).

Module structure:

```
src/thunk_lexer.yrl, thunk_parser.yrl  # tokens and grammar
lib/thunk/eval.ex                      # interpreter
lib/thunk/primitives.ex                # the thirteen primitive operations
lib/thunk/scheduler/{sequential,local,distributed}.ex
lib/thunk/worker.ex                   # deque, steal loop, program cache
lib/thunk/cluster.ex                  # submitting jobs to the cluster
lib/thunk/membership.ex                # seeds and gossip of the member list
lib/thunk/dashboard.ex                # collector and HTTP server
lib/mix/tasks/thunk.demo.ex            # command-line demonstrations
priv/prelude.thunk                    # standard library, in Thunk
priv/demos/*.thunk                    # demonstration programs, in Thunk
```

5 Validation

5.1 Automatic Testing

Test infrastructure. Tests are written with ExUnit and cover the parser, the interpreter, the library, the schedulers, the worker and the cluster. Three techniques are used throughout:

- **Oracle testing:** the sequential scheduler is the reference. Every program must give identical results under the other schedulers, and the demonstration programs

are compared with native Elixir implementations on random inputs.

- **Real multi-node tests:** the test process turns itself into a distributed node and starts additional Erlang nodes with the OTP `peer` module, each running the Thunk application and connecting back. These tests exercise the actual protocol between nodes, not a simulation.
- **Fault injection:** tests start fake thieves that take a piece and die, or report a program error, and multi-node tests stop a real node in the middle of a job.

Test coverage. Table 3 maps the requirements of section 2 to the tests that check them.

How to run. The whole suite, including the multi-node tests, runs with `mix test`; formatting and warnings are checked with `mix format --check-formatted` and `mix compile --warnings-as-errors`. The multi-node tests require the Erlang name service (`epmd`), which the test helper starts if needed; if distribution cannot be started, they are skipped with a message.

5.2 Experimental Evaluation

Setup. The first environment is a laptop with 12 hardware threads running Windows 11 and Docker Desktop, with three Thunk nodes in containers and a fourth node, inside the first container, that submits jobs without computing any piece itself, so that all work below the first split has to be stolen. The second is the laptop together with a virtual private server with 4 cores running Debian, about 22 ms away and also hosting other services; to make the two machines comparable, the laptop node was limited to 4 schedulers, and a third node on the laptop submitted the jobs. The server was reached through an SSH tunnel bound to its loopback interface, so no port was exposed to the Internet, and automatic full-mesh connection was disabled so that the server never tries to connect to the private addresses of the laptop. Four workloads were used: mergesort of random integers, word count of random text, a raytraced cube and the Mandelbrot set, both images computed row by row in fixed-point arithmetic. Times and counters come from the output of the `demonstration` command. Since the containers of the first environment share the processors of one machine, that environment measures the *overhead* of distribution rather than speed-up; the second measures the effect of adding a machine.

Correctness. Every run of every experiment reported a correct result: sorted lists equal to the native sort, word counts equal to the native count, and images identical byte for byte whatever the scheduler, the number of nodes and the size of the pieces (fig. 6).

Overhead and speed-up. Figure 7 compares the schedulers on one machine and the cluster on one and on two machines. On the laptop, parallel execution is four to five times faster than sequential execution, and the cluster of three containers costs nothing

Requirement	What is checked	Test module	Kind
FR1	every special form, scoping, order of evaluation and the error for each misuse; a loop of a million iterations in constant memory; list, sorting and parallel library against native results	parser, eval, primitives, prelude	unit, oracle
FR2	base case, split and merge order, nested dc , errors for a threshold that is not a boolean or a split without two parts	scheduler/sequential	unit
FR3	oldest piece to thieves, take-back, counters; a mergesort submitted on a node that computes nothing returns the sequential result and the other nodes report steals	worker, cluster	unit, multi-node
FR4	a job using a definition that exists only on the submitter completes on the cluster; programs are fetched by job and cached	cluster, worker	multi-node
FR5	an error in a piece computed on another node is raised at the submitter; a program error reported by a thief is not retried	cluster, scheduler/distributed	multi-node, fault injection
FR6	a fake thief takes a piece and dies and the owner solves it; a real node with high capacity is stopped in the middle of a job, the result is exact and the survivors report recovered pieces	scheduler/distributed, cluster	fault injection, multi-node
FR7	mergesort, word count and both images give identical results under the sequential, local and distributed schedulers, and equal native implementations	demo, cube, mandelbrot	oracle
FR8	with full mesh disabled, B seeded with A and C with B all end up connected; a node joining during a job takes part in it	membership	multi-node
Determinism	the same result for every scheduler and any local capacity	scheduler/local, demo	oracle
Observability	rates from counters, a node that disappears shown as down, page and data served over HTTP	dashboard	unit, multi-node
Piece size	the functions carried by the pieces of pmap do not capture the input list	prelude	unit

Table 3: Requirements and the tests that check them. Module names are relative to **test/thunk/**; multi-node tests start real additional nodes, fault injection tests kill a thief or a node during a job.

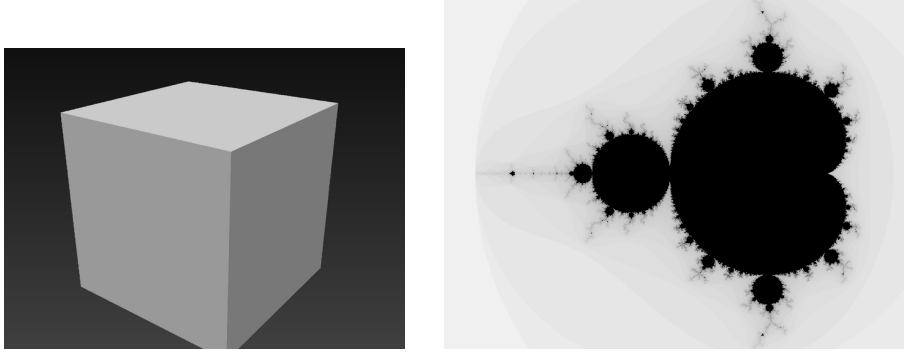


Figure 6: The two image workloads, the raytraced cube at 1280×960 and the Mandelbrot set at 640×480 , both computed by the laptop and the server together.

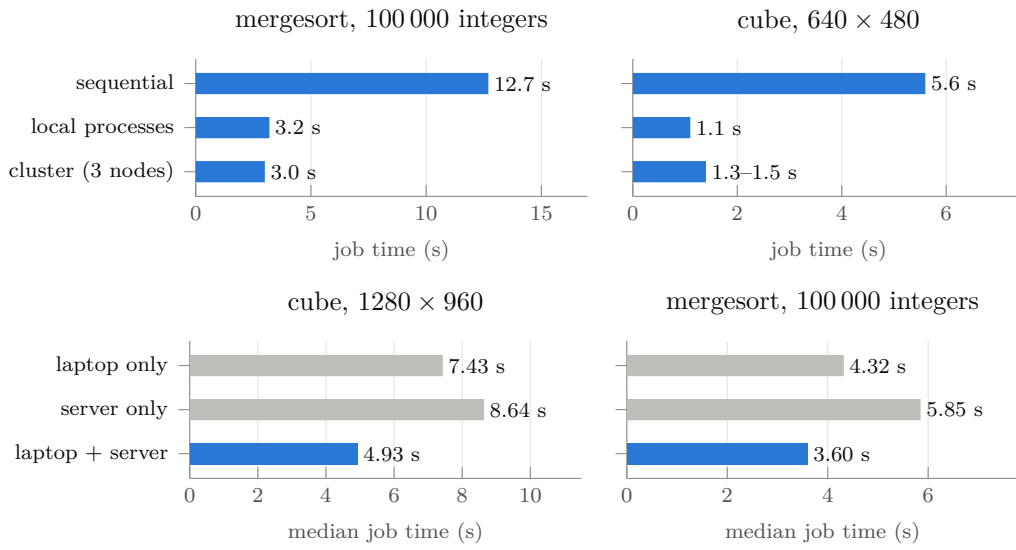


Figure 7: Job time. Top: the three schedulers on the same machine, so that the difference between the cluster and local processes is the cost of distribution. Bottom: median of five runs with one machine and with two machines 22 ms apart, each running a node with four schedulers.

measurable on the sort and between a fifth and a third more time on the small image: this is the price of stealing, copying pieces between containers and returning results. On two machines, with five runs per configuration after a warm-up run, the second machine makes jobs faster, the image 1.5 times and the sort 1.2 times; the server alone is slower than the laptop alone, since it also serves other applications and every piece it computes crosses the network. The division of work between the machines was not configured anywhere and adapts to the cost of communication: the server took more than half of the pieces of the image, which are cheap to send and expensive to compute, but only a quarter of the pieces of the sort, which carry the list they sort across the 22 ms link.

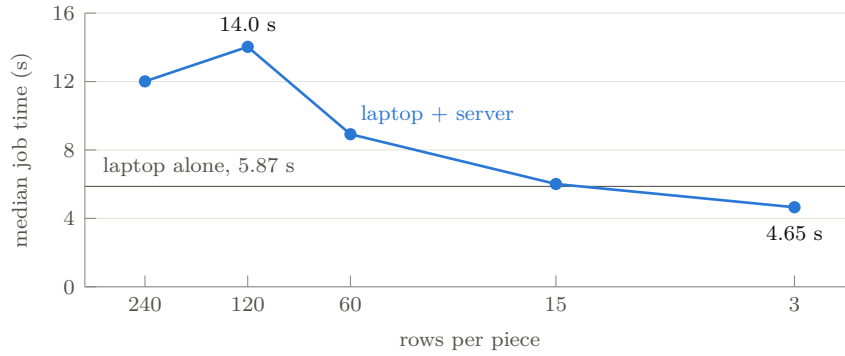


Figure 8: The Mandelbrot image computed by the laptop and the server together, with pieces from 240 down to 3 rows; median of three runs. The reference line is the laptop alone with small pieces.

Irregular work. In the workloads above, pieces of the same size cost about the same. Work stealing is designed above all for the opposite case, in which nobody can tell in advance which pieces are expensive: in the Mandelbrot image the rows that cross the set cost up to 17 times more than the rows far from it, while the rows of the cube differ by a factor of two. Figure 8 shows the image computed on the two machines with pieces of different sizes. With a few large pieces the two machines are more than twice as slow as the laptop alone: each piece is solved by a single process, most processors stay idle, and the job waits for whoever received an expensive middle piece, in the worst case the slower machine. As the pieces become smaller, idle processors keep stealing, the expensive rows are spread over both machines, and with pieces of three rows the two machines beat the laptop alone. The cube, whose time hardly changed with the size of the pieces, shows the contrast: when the cost of the work is uneven, dividing it finely and letting idle machines take it is what balances it, without anyone knowing where the expensive parts are.

Granularity. The threshold decides how many pieces a job is divided into, and therefore how much work there is to share and how many steals are needed. Figure 9 shows mergesort of 100 000 integers on the cluster at four thresholds. With coarse pieces there are too few of them to keep all nodes busy and the job is almost as slow as a sequential one; with very fine pieces the cost of creating, exposing, stealing and combining thousands of pieces exceeds the useful work and the job becomes slower than a sequential one; the best results lie in between. The number of steals, instead, stays between 14 and 33 while the pieces grow from 31 to almost two thousand, as predicted for randomized work stealing [1]: thieves take the largest piece available, so a few steals give each node a large share of the work, which it then splits and computes locally. The word count workload adds a caveat: the parallel part of a job must last long enough for thieves to arrive, and on a small text the sequential splitting into words dominated and nothing was stolen.

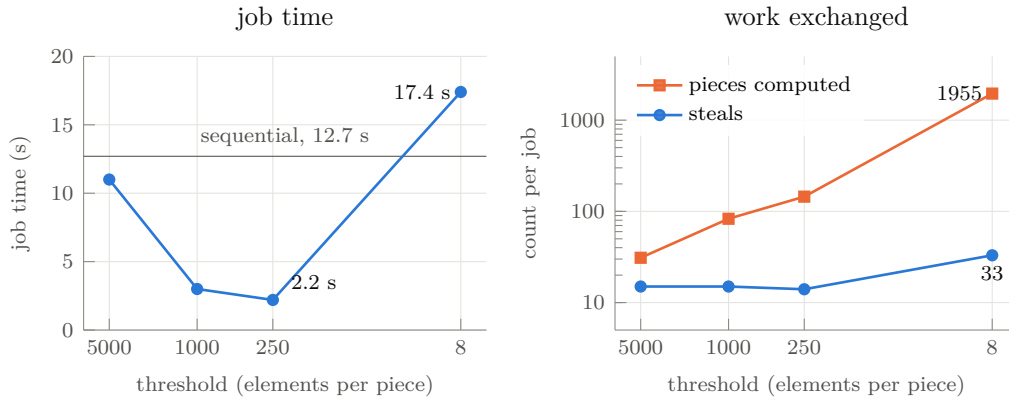


Figure 9: Mergesort of 100 000 integers on the cluster, from coarse pieces (left) to very fine pieces (right). Left: job time, with the sequential time as reference. Right: pieces computed by the nodes and steals needed, on a logarithmic scale.

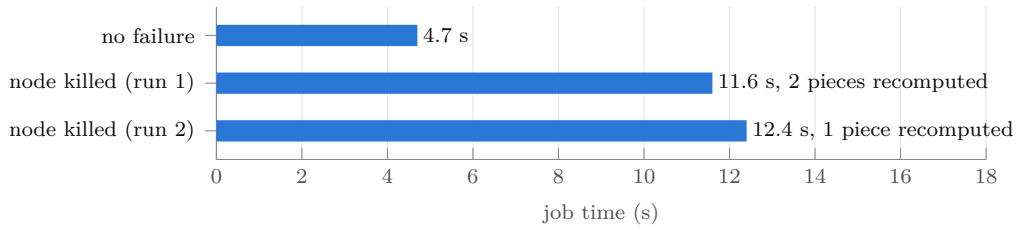


Figure 10: Rendering of a 1280×960 image on the cluster, without failures and with a node killed after two seconds. In both runs with a failure the image is identical to the reference.

Failure recovery. A crash was simulated by killing one of the three computing containers two seconds into the rendering of a 1280×960 image (fig. 10). In both runs the job completed and the image was identical, byte for byte, to the one rendered without failures; the lost pieces were recomputed by their owners. Recovery is *lazy*: the loss of the node was logged about eight seconds before the owner of a lost piece noticed it, because an owner looks at a stolen piece only after finishing its own half, so the job waited for work that no longer existed. The same experiment on the two machines, killing the server node three seconds into the job, completed in 6.6 and 10.5 s instead of about 4.9 s, again with identical images; there the loss was detected as soon as the connection closed, because the owner of the first lost piece was already waiting for it.

5.3 Acceptance Testing

Manual acceptance testing was performed on the Docker Compose cluster, for scenarios that depend on the deployed system as a whole:

- **Demonstrations:** the four workloads were submitted from a node inside the cluster, checking that every run reports a correct result and that the counters of all nodes change; the rendered images and a ten-frame orbit animation around the

cube were inspected.

- **Node crash:** a container was killed during a job with `docker compose kill`, and the result compared with a run without failures (fig. 10).
- **Dashboard:** the live page was watched while jobs ran and while a node was killed, checking that the rates move and that the node is marked as down.

These scenarios need Docker and visual inspection; their automated counterparts are the multi-node tests.

6 Deployment

6.1 Requirements and Setup

The cluster needs Docker and Docker Compose; running a node or the tests outside containers needs Elixir 1.20 and Erlang/OTP 29, which on Windows without administrator rights can be installed with Scoop. A single image is built from the repository: it compiles the project and starts a node whose name, cookie and peers are taken from environment variables.

```
git clone <repository url> && cd thunk
mix compile && mix test           # outside containers
docker compose up -d --build     # the cluster
docker compose logs
```

6.2 Docker Compose Design

The Compose file declares three services, `node1`, `node2` and `node3`, built from the same image (fig. 11). Each service has a host name equal to its service name, lists the other two as peers and shares the same cookie. Short node names are used, because the Erlang runtime rejects long names whose host has no dot, as is the case for container host names, and nodes retry their seeds every second until one answers, so they can start in any order. A fourth service, `joiner`, is disabled by default (Compose profile `join`); it has no fixed host name and only `node1` as seed, so it can be scaled to add any number of nodes to the running cluster. The submitting node and the dashboard are short-lived nodes started inside the first container, whose port 4000 is published to the host.

After startup, each node logs the other nodes as they connect and as they become members; when a job is submitted, the counters of the nodes show pieces stolen and evaluated; when a node is stopped, the others log its disconnection.

7 User Guide

7.1 Writing a Program

A program is a sequence of definitions with a `main` of one argument, and may use every function of the prelude. For example, the sum of squares of a list computed in parallel:

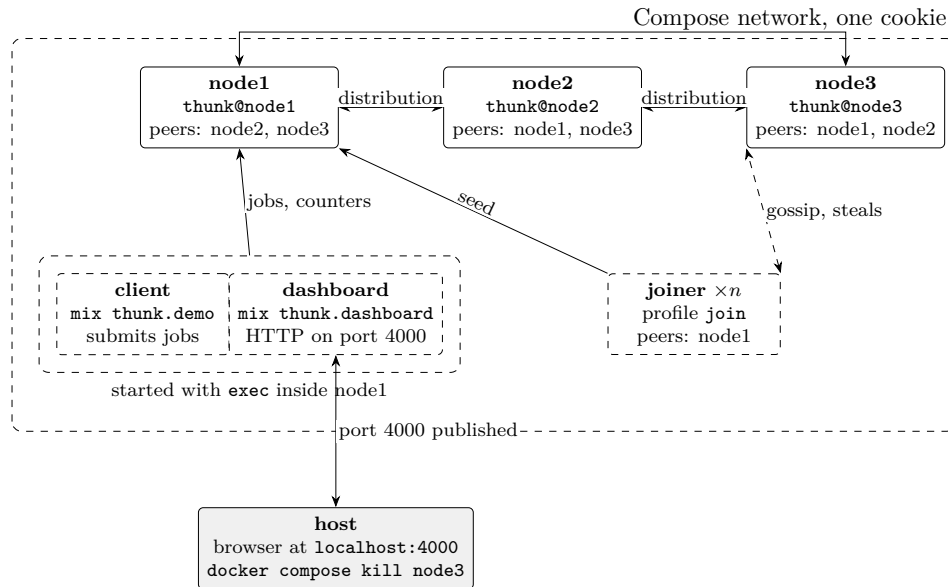


Figure 11: Docker Compose deployment. Three permanent services form the cluster; the optional joiner service can be scaled to add nodes; the submitter and the dashboard run as short-lived nodes inside the first container. Dashed boxes are optional or short-lived.

```
def square(x) = x * x
def small?(v) = length(v) < 1000
def main(xs) = reduce(add, pmap(square, xs, small?), small?)
```

Functions passed to parallel operations should be defined at top level, as here, so that pieces of work carry only the data they need.

The program is submitted from an Elixir shell on any node:

```
ctx = Thunk.Cluster.load(File.read!("sum.thunk"))
Thunk.Cluster.run(ctx, Enum.to_list(1..100_000))
```

7.2 Running the Demonstrations

```
docker compose exec node1 sh -c 'elixir --sname client \
  --cookie thunk-demo -S mix thunk.demo cube --width 640 --height 480'
```

The demonstration command accepts `mergesort`, `wordcount`, `cube` or `mandelbrot`, and options for the size of the input, the threshold, the number of repetitions, the scheduler, and the number of pieces the submitting node may compute itself. The cube accepts `--frames` to render an orbit as an animated image. Each run prints its time, the outcome of the check, and the counters of every node:

```
peers: [:thunk@node2, :thunk@node3]
run 1: cube 640x480 angle=3798 threshold=8 ok 1253 ms
```

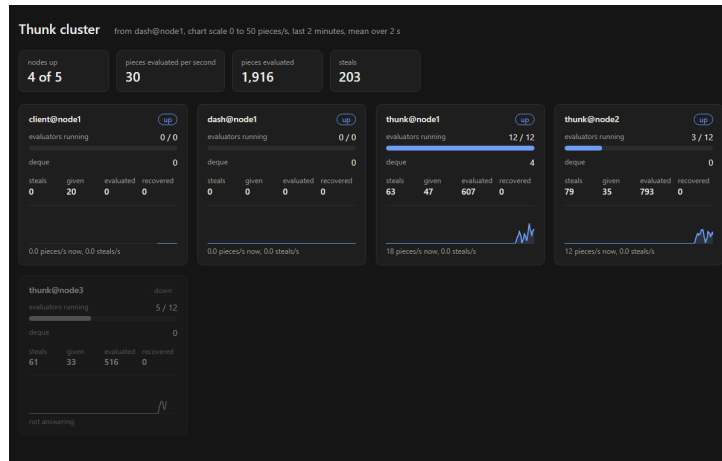


Figure 12: The dashboard during the rendering of a 1280×960 image on the Compose cluster, a few seconds after `node3` was killed: the two surviving nodes are computing, the submitter and the dashboard node compute nothing, and the lost node stays on the page as down.

```
client@node1: steals=0 stolen_from=3 evaluated=0 recovered=0
thunk@node2: steals=20 stolen_from=9 evaluated=151 recovered=0
...
```

Adding nodes to a running cluster. The following command starts two more nodes, which join through `node1`, learn the other members by gossip, and take work from jobs already running:

```
docker compose --profile join up -d --scale joiner=2
```

Watching the cluster. The dashboard is started inside the first container:

```
docker compose exec node1 sh -c 'elixir --sname dash --cookie thunk-demo \
  -S mix thunk.dashboard --port 4000'
```

The page at `http://localhost:4000` shows one card per node, with its running evaluators, the length of its deque and its counters, and the rate at which each node computes pieces over the last two minutes. A node that stops is greyed out and marked as down (fig. 12).

Simulating node failures. While a long job is running, a node can be stopped from another terminal:

```
docker compose kill node3
```

The job completes, and the `recovered` counters show which nodes recomputed the lost pieces.

8 Self-evaluation

This project was entirely designed and implemented as an individual effort by Yosberto Baro Carbonell.

8.1 Strengths

- **Fully decentralized architecture:** no master, no central queue, no shared store; pieces are stolen between computing nodes, not only from the submitter.
- **Computations as data:** nodes do not need to know a program in advance, and no compiled code is ever exchanged.
- **Fault tolerance without replication:** node crashes are tolerated by recomputation, and results are always identical to a sequential reference.
- **Testing with real nodes:** automated tests start real additional nodes and stop them in the middle of jobs; the cluster is reproducible with Docker Compose.

8.2 Weaknesses and Limitations

- **Small-scale evaluation:** most experiments ran on one laptop, and the multi-machine ones on two machines, one shared with other services; scaling beyond two machines has not been measured.
- **Lazy recovery:** the owner of a lost piece notices the loss only when it needs the result, which can be long after the node failed.
- **No tolerance of the submitting node's failure:** it holds the root of the job.
- **Closures carry their whole environment:** functions written inside other functions may carry unused data with every piece.
- **Interpretation cost:** much slower than native code, a deliberate trade-off that makes computations mobile and keeps the runtime independent of the programs.
- **No security:** the system assumes a trusted network; across the Internet it was protected by an SSH tunnel, not by the system itself.

8.3 Overall Assessment

The project achieves the objectives of the approved proposal, a combinator API with recursive decomposition, decentralized work stealing and a multi-node Docker Compose demonstration, and extends them with recovery from node failures, dynamic membership and a live dashboard. Its main lesson is how the choice of the computation model removes problems that would otherwise need dedicated protocols: termination detection is unnecessary with fork-join, and consistency of results follows from purity.

9 Future Works

- **Larger evaluation:** more than two dedicated machines, to measure how the speed-up grows with the number of nodes and how the division of work depends

on the network.

- **Proactive recovery:** a worker already knows which thief took each of its pieces; it could monitor thieves and put lost pieces back in its deque as soon as a node fails, instead of waiting for their owners to notice.
- **Capture of used variables only:** closures could capture only the variables their body uses, making every piece minimal; since this changes when two functions are equal, it is a decision about the language.
- **Improved stealing policies:** stealing half of a victim's deque, or choosing victims according to their load, are known refinements of randomized work stealing.
- **Membership at larger scale:** a larger cluster would need partial views, where each node knows only a sample of the others instead of all of them.
- **Parallel text processing:** splitting the text of the word count inside the parallel part of the job, to distribute tokenization too.
- **Native code generation:** translating programs into Erlang modules compiled and loaded at run time. Compiling closures into trees of Elixir functions was tried and was no faster than the interpreter.
- **Language extensions:** floating-point numbers, list literals, and error messages that point to the line of the source where a run-time error happened.
- **Security:** running the Erlang distribution over TLS, for clusters that cannot assume a trusted network.
- **Termination detection:** needed by an asynchronous task model, where no process waits for the pieces it creates, for example to reclaim the work of a job whose submitter failed.

References

- [1] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.
- [2] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. In *6th Symposium on Operating Systems Design and Implementation (OSDI)*, pages 137–150, 2004.
- [3] Edsger W. Dijkstra and Carel S. Scholten. Termination detection for diffusing computations. *Information Processing Letters*, 11(1):1–4, 1980.
- [4] Erlang/OTP. Distributed Erlang. <https://www.erlang.org/doc/system/distributed.html>. Accessed 22 September 2026.
- [5] Matteo Frigo, Charles E. Leiserson, and Keith H. Randall. The implementation of the Cilk-5 multithreaded language. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 212–223, 1998.
- [6] Stephen C. Johnson. Yacc: Yet another compiler-compiler. Computing Science Technical Report 32, Bell Laboratories, 1975.
- [7] John McCarthy. Recursive functions of symbolic expressions and their computation by machine, part I. *Communications of the ACM*, 3(4):184–195, 1960.